

Кейс 1: «Авито Тамагочи»

Мы в Авито хотим повысить вовлеченность всех пользователей и сформировать у них новую привычку — возвращаться к нам снова и снова. Наша идея — добавить геймификацию «поверх» основного продукта. Это виртуальный питомец, за которым пользователь ухаживает и который растет вместе с его активностью.

Отличие от классического тамагочи в том, что питомец должен давать реальную пользу клиентам Авито. Улучшение питомца и уход за ним — не самоцель, а путь к бонусам, которые можно применить в основном продукте.

Бизнес-задача

Создайте MVP веб-приложения, которое:

Формирует привычку возвращаться

У пользователя есть ежедневный повод зайти в приложение.

Игрок заботится о питомце, следит за его состоянием, выполняет простые действия и прокачивает уровень. Чем регулярнее пользователь взаимодействует с питомцем, тем заметнее прогресс.

Награждает за регулярность

Чем активнее пользователь, тем быстрее растёт питомец и тем ценнее бонусы и награды.

Имеет связь с брендом и основным продуктом

Авито это в первую очередь доска объявлений. Это значит, что визуал должен быть связан с дизайном Авито, а награды - мотивировать делать основные действия, которые предоставляет доска объявлений.

Что нужно реализовать

Создайте пользовательский сценарий.

Пользователь регистрируется, получает питомца и начинает о нём заботиться. Его действия влияют на состояние питомца и увеличивают опыт. Накопленный опыт повышает уровень, а уровень открывает доступ к наградам. Награды могут быть разного характера, все заканчивается лишь вашим воображением, но не забывайте про связь с брендом!

Сформулируйте и опишите документацию

Опишите какой пользовательский путь вы проектируете, какие параметры и действия вы выбираете и почему. Важно расписать, как работает прогрессия (уровни, опыт).

Проработайте прогрессию и награды

Продумайте экономику приложения. Нужно ответить для себя на вопрос «Как мне мотивировать пользователя посещать мою игру, чтобы затем конвертировать эти заходы в конверсию на Авито?». Мотивация может быть разной, например, промокоды (бесплатная доставка, автотека), скидки (в определенной категории, на доставку, на другие продукты на Авито).

Механика выдачи наград остается на ваш выбор (за уровень, за серию регулярных действий, за достижения). Важно: пользователь должен понимать, что нужно сделать, чтобы получить конкретный бонус, и видеть прогресс на пути к нему.

Главное правило бонусов и наград: они привязаны к конкретному пользователю. Нужно продумать, как сделать это безопасным, чтобы не было возможностей обхода системы и использования одной награды несколькими лицами.

Отображение состояния питомца в реальном времени

Реализуйте возможность через WebSockets

Лидерборд

Покажите, как пользователь сравнивает себя с другими — по уровню, опыту или комбинированному показателю. Формат и механику определите сами.

Ежедневная сводка

Пользователь должен получать обратную связь о своём дне: что изменилось у питомца, какой прогресс, какие награды заработаны. Формат — на ваше усмотрение.

Важно: вам нужно не просто написать приложение, а решить бизнес-задачу.

Почему пользователь захочет вернуться завтра? Что он увидит? Что изменится у питомца? Как он поймет, что его действия имеют значение?

Сильное решение — это не столько код, сколько ваша продуктовая логика.

Требования по стеку

Frontend

- React 18+
- Typescript

Разрешено использование любых внешних библиотек, в том числе:

- Дизайн-система/UI-kit (Material-UI, Ant Design)
- Стейт-менеджмент (Redux, MobX, Effector)
- Линтер (ESLint)
- Prettier
- Система сборки (Webpack, Vite)
- Библиотека для работы с асинхронными HTTP-запросами (React Query, Axios)

Backend

- Язык программирования – Golang
- База данных — выбрать самостоятельно, к выбору приложить обоснование, почему выбрана та или иная БД
- Деплой — приложение поднимается через Docker Compose
- Описать конфигурацию линтера (.golangci.yaml в корне проекта)

Оформление решения

- Общий репозиторий с frontend + backend
- Приложение поднимается через docker-compose
- Описанный README файл

Использование ИИ

- Вы можете пользоваться нейросетями, но делайте это прозрачно. Укажите дополнительно — на каких этапах и каким образом вы ими пользовались
- Если сможете интегрировать ИИ в ваш MVP — это станет плюсом при оценке вашего решения

Как жюри будет оценивать ваш результат

По каждому пункту можно получить от 0 до 5 баллов.

- **Обязательная бизнес-логика проекта**

Отличный результат: основной сценарий кейса работает полностью - пользователь может пройти путь от начала до конца, все ключевые состояния предусмотрены, логика соответствует бизнес-задаче.

- **Реализованный доп. функционал**

Отличный результат: дополнительные функции усиливают бизнес-задачу, органично выглядят в сценарии и не мешают обязательному MVP.

- **Наличие README**

Отличный результат: README позволяет запустить и проверить проект без устных пояснений команды; описаны запуск, зависимости, сценарии, API, ограничения MVP и ключевые решения.

- **Наличие тестов**

Отличный результат: покрытие юнит-тестами самых важных участков кода.

- **Наличие линтера**

Отличный результат: наличие на бэкенде и фронтенде линтеров с правилами и их обоснованием.

- **Дизайн и архитектура приложения**

Отличный результат: интерфейс понятный и аккуратный, архитектура проекта логичная, части приложения разделены по зонам ответственности.

- **Продуктовые решения, позволяющие достичь бизнес цели**

Отличный результат: понятное обоснование продуктовых решений, которыми достигается бизнес-цель.

- **Чистота и качество кода, консистентность стиля**

Отличный результат: код читаемый, структурированный, с понятными названиями, без мертвого кода и костылей. Настроены инструменты автоматической проверки качества.

- **Доступность в интернете**

Отличный результат: проект развернут и доступен по ссылке, жюри может проверить основной сценарий без локального запуска.